2019 IEEE 5th Intl Conference on Big Data Security on Cloud (BigDataSecurity), IEEE Intl Conference on High Performance and Smart Computing (HPSC), and IEEE Intl Conference on Intelligent Data and Security (IDS)

Security Vulnerabilities in Consumer IoT Applications

Arjun Shakdher
Dept. of CIT
Purdue University
West Lafayette, IN, USA
arjunshakdher@purdue.edu

Suyash Agrawal
Dept. of CIT
Purdue University
West Lafayette, IN, USA
agrawa80@purdue.edu

Baijian Yang
Dept. of CIT
Purdue University
West Lafayette, IN, USA
byang@purdue.edu

***Abstract*—Extensive penetration testing to IoT applications was conducted in this work to discover the vulnerabilities. In addition to the study the most vulnerable security flaws defined by the Open Web Application Security Project (OWASP), this work also tested a set of man-in-the-middle attacks exploiting found vulnerabilities. It is discovered that a wide range of IoT apps in smart homes, security system, health-care and connected cars are susceptible to a set of attacks, and some of those apps have over 1 million downloads. Countermeasures were proposed as recommendations to secure the apps for IoT devices.**

***Index Terms*—IoT, Application Security, Vulnerability, Man-in-the-middle attack, Penetration Test**

I. Introduction

We are living in a world that is ongoing a massive phase of digitization. According to Gartner [1], there was an increase of 31 percent in the number of connected devices in 2017 compared to the previous year and it is estimated that around 20 billion Internet of Things (IoT) devices will be added to the current network by 2020. Smart homes and wearables are the two most popular areas where IoT exists [2]. Since more and more things in close proximity to us are being exposed to the threats prevalent on the Internet, it is a matter of increasing concern to secure these devices. Manufacturers all around the globe across various crucial industries are trying to create connected devices and launch it to the market as quickly as possible in order to gain the market share. As a result of this hurry, there is a triple constraint of security, cost and time. The tightly coupled attributes create a trade-off where cost and time get priority and quality are not of primary concern. Due to poor quality, security is compromised greatly.

There are also various hardware limitations. IoT devices generally do not possess very high computing power. The use of low powered micro-controllers implies low-security features. Whenever such devices are added to the network, they create a security bottleneck and attackers will always go for the weakest link in the chain. The purpose behind these attacks can range from company secrets to blackmailing to theft.

IoT can be broadly categorized into 3 layers - Application, Network, and Physical. While the physical and network layers are responsible for device identification and communication/connectivity respectively, the service-oriented application layer is responsible for ensuring the back-end and end-user applications function as expected while being secure [3]. The impact of poor application security is huge when it comes to crucial industries where there is no tolerance for a security breach. In 2016, attackers used tens of millions of IoT devices to launch a Distributed Denial of Service (DDoS) attack on the US-based DNS provider, Dyn [4]. Authorization and authentication that were being securely achieved using established standards earlier is now a challenge because of the huge spectrum of IoT devices on the network [5]. Even though research has been conducted to tackle this problem, a common standard is not present [6] [7]. Another issue with the connected devices infrastructure is that if one of the devices is compromised, it poses a threat to other devices in the network [5]. While there are programs that can help analyze software for vulnerabilities before release, the variety of different platforms used by them makes it difficult to scan and creates the need for custom development [8] [9]. The big set of heterogeneous devices has become a factor for newer security concerns in addition to the severe existing ones [5].

This paper introduces various attacks vectors that can be performed on some consumer IoT device applications; documents the results of the attacks performed, and provides a summary and suggest possible solutions to strengthen IoT application security. All penetration testing experiments were performed in a controlled test environment.

The remainder of this paper is organized as follows. Section II presents the relevant related work. Section III describes the methodology, the functioning and the technologies used. Section IV shows the experimental setup, Section V presents the implementation followed by the testing results under various scenarios in Section VI. Section VII explains the findings and provides counter-measures to strengthen IoT security and Section VIII concludes the paper.

II. Literature Review

There has been a lot of quality research in the IoT security domain but most of it is focused on the actual devices itself. Security challenges and security mechanisms are the major themes of investigation for researchers which generally include

This work is partially funded by Purdue Polytechnic Holistic Safety and Security Research Impact Area.

978-1-7281-0006-7/19/$31.00 ©2019 IEEE
DOI 10.1109/BigDataSecurity-HPSC-IDS.2019.00012

a specific type of threat based on an objective [12] such as, attacks on RFID systems [11] or wireless network systems [10] as mentioned earlier in the introduction. IoT attacks have been broadly categorized into 4 types - physical, network, software and encryption attacks [12].

Atzori et al. [13] state that IoT is extremely vulnerable to attacks mainly because of three reasons i.e components are unattended for most of the time making it prone to physical attacks; communication is majorly wireless thus it easy to eavesdrop; most components are low capability in terms of energy and computing resources thus cannot implement complex security schemes. They also talk about sensor nodes as being nodes of the Internet and thus need authentication. They observe that none of the existing solutions can help in solving the proxy attach problem i.e Man-in-the-Middle (MITM) attack. A MITM attack is when an attacker is able to get access to the connection between two nodes and might use this for eavesdropping or other malicious use [14]. This leads to a major data integrity issue. Even in RFID systems, the password length supported by most tag technologies is too short to have a considerable amount of protection. Though the authors briefly mention the idea of using cryptographic algorithms it is observed that these low resource and low power devices cannot handle such complex algorithms [13].

Zhang et al. [5] identify authentication and authorization, privacy, and lightweight cryptosystems as major challenges in IoT security. Although public key cryptosystems are widely available, the use of such systems is unfeasible due to lack of global root certificates. Thus, authentication is delegated which leads to a potential challenge in terms of security. Furthermore, the privacy aspect has two major challenges i.e. data collection policy and data anonymization. Though the privacy of data collection can be ensured by having adequate policies in place, data anonymity requires a high level of cryptographic protection and concealment. Owing to the vast variety of devices that work on shared information, ensuring a high level of privacy is highly desirable. The researchers observe that this can be achieved by applying homomorphic encryption. But due to computational overheads of using these public-key cryptosystems, the need for lightweight security protocols is a major challenge for IoT security. [5]

An attacker model is introduced and applied to both centralized and distributed systems and the main challenges are then studied [15]. Software vulnerabilities are identified as the main source of security vulnerabilities in a system [12]. Phishing attacks, viruses, malicious scripts, and DoS attacks are common attacks that have been performed on software applications [16].

Sicari et al. [17] mention that with reference to data security, data anonymity, confidentiality, and integrity need not be guaranteed along with authentication and authorization mechanisms which can restrict access to any kind of unauthorized users to the system. The authors focus on the importance of embedded security in the devices which can handle the data and process it in compliance with user needs and rights.

Alaba et al. [18] investigate application threats in multiple smart application domains like smart city, smart homes, and smart health-care. They explore Denial of Service (DoS), eavesdropping, device end-point, counterfeiting, and MITM attacks. They observe that small IoT devices like smart-watches and smart bulbs systems are fragile, where an active attacker can easily manipulate the contents due to the light-weight nature of the security of these devices.

In [19], various intruder detection techniques are surveyed in order to define a common reference of key terms between intruder detection, artificial intelligence, and the Internet of Things. It concludes that the inclusion of daily-use devices (objects) which can connect to network applications opens further avenues for attacks and exploitation.

It is important to note that the basic gap in the majority of the research done to date is the inclination of the researchers towards the physical devices as a whole. Most of the authors mentioned above are focusing over securing the physical devices from various possible vulnerabilities, but this paper is more concerned about how the communication of these physical devices with their web and mobile applications is also prone to a large number of attacks and can affect the whole system.

This study was focused on the application security aspect of IoT as the system is only as strong as its weakest chain and the application side is a highly vulnerable target.

III. Methodology

The methodology for this paper includes penetration testing IoT applications and finding out their vulnerabilities. These attacks were executed under a chosen test environment. The results are documented and a summary of results is compiled. Moreover, possible ways to prevent attacks are suggested.

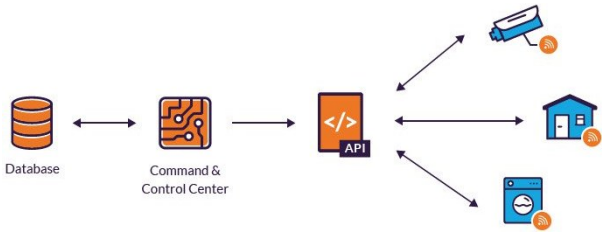

Fig. 1. IoT Device management framework

As shown in figure 1, IoT devices need to be managed internally and externally. Each device is connected to a management unit i.e. a Command and Control (C&C) center. The interactions between the management unit and the devices are enabled using application programming interfaces (API). Once connected, the devices can communicate with other devices and applications. This centralized nature of C&Cs and APIs creates a window of weak spots that can be easily exploited.

An application can be attacked via many different paths. These paths can vary in the level of ease to find and exploit the application. The possible harm can typically range from being non-consequential to major loss such as shutdown. In order to be able to evaluate the level of security that an application

exhibits, different attacks were executed for each case. It is important to note that each application is unique in itself, so the level of threat from particular attacks will also differ in each case. After these attacks were carefully executed, a survey of robustness of all the applications tested with respect to these attacks was compiled.

A list of applications was compiled and penetration tested based on several factors including the number of people they impact, nature/domain/industry of application and the consequence of their impact.

In conjunction with Open Web Application Security Project's (OWASP) most vulnerable security flaws [20], attack vectors that were performed for compromising the security of the IoT applications fall across the following domains:

1) Code Injection. *e.g.* SQL injection.
2) Broken authentication. *e.g.* credential stuffing, password oracles, and improper session management.
3) Sensitive data exposure. *e.g.* Applications that do not use Transport Layer Security (TLS) or have weak authentication can be attacked to intercept the request, steal cookies and replay them, modify them and gain access to user's private data.
4) Broken Access Control. *e.g.* Data not verified in SQL call; Force browser to directly open administrator or unauthorized URL.
5) Security misconfiguration. *e.g.* Exploit applications bundled with the application server that have the default credentials; reverse engineer applications with directory listing enabled; and exploit information in the error handling framework and logs.
6) Reflected XSS (Cross-site scripting) *e.g.* Enter the code into the input field of a web application that goes unvalidated and is eventually executed by the browser.

Parallel to the above attacks, penetration testing using popular techniques such as Domain Name Server (DNS) Lookups, MITM, DNS Cache poisoning and Denial of Service (DoS) was performed.

IV. Experimental Setup

The testing used open-source tools available across platforms. A non-exhaustive list of the tools is given below:

- MetaSploit - exploit development and penetration testing.
- Bettercap v1.6.2 - a suite of tools for MITM attack.
- SSLStrip+ v0.4 - Protocol downgrade attack.
- ARPSpoof - ARP spoofing.
- Wireshark, Burpsuite - Network protocol analyzer.
- NMAP, ZenMAP - Network scanning.
- MIMTf v0.9.8 - Man-in-the-Middle framework.
- HTTPS Proxy - Redirecting traffic.
- DNSChef v0.4 - Custom DNS Server.
- Kali-Linux Tools - Pre-installed pen-test suite.

These tools were used to check the vulnerability of various applications and a detailed list of findings is provided in the Results section.

V. Implementation

The authors considered a scenario where the victim is using several IoT applications that are communicating to their respective servers for controlling their IoT devices. It is assumed that the penetration tester is on the same network as the victim. The penetration testing of these applications began with information gathering.

First, a network discovery was run on the entire network using NMAP and ZenMAP to find out all devices and hosts connected on the network, what applications they were running, the operating system and if there were any firewalls in use. This scan revealed information about our victim.

Next, sniffing over the wire was performed using Wireshark, Bettercap, and Burpsuite. Wireshark is one of the best protocol analyzers that helps look at the content of the communication between the app and server at various levels.

After the basic network analysis, injection attacks were performed. Following that, a MITM attack was planted where the attacker was placed between the victim (running the IoT application) and the server [21]. MITM attacks can be performed to either impersonate another entity or simply eavesdrop. All this is done in a way such that the conversation between the two parties works as expected. The goal of such attacks can range from sniffing passwords to stealing confidential information to blackmail. As seen in figure 2 [21], the MITM attack in this paper was carried out in two phases.

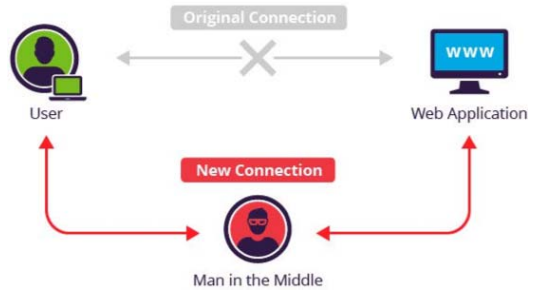

Fig. 2. Man in the Middle Attack

The first phase involved the interception of traffic through the attacker's computer before it arrives at the location it was intended for. This was done using the Address Resolution Protocol (ARP) spoofing. Communication takes place through routers by sending packets to the intended Media Access Control (MAC) addresses at the data link layer. ARP is a communication protocol that maps IP addresses with MAC addresses. When the victim running the IoT application communicates with the server, it broadcasts an ARP packet for finding the MAC address that is mapped to the destination (server's) IP address. Usually, the destination server would respond to the packet providing it's MAC address. The weakness of ARP cache is that it has a short-term memory where it only remembers the last ARP reply that is sent to it without confirming the identity of the sender. At this point, the attacker bombards the victim with a spoofed ARP reply to the victim

Fig. 3. ARP spoofing

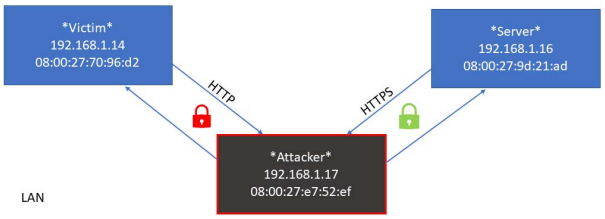

Fig. 4. Downgrade HTTPS to HTTP using SSLStrip

and thus the victim is fooled to believe the true identity of the attacker. After this, the attacker is successfully placed in between the victim and the server forwarding packets to each other without both partying knowing anything about it. This is illustrated in figure 3 [22].

All the traffic passing through the attacker's machine was analyzed by Wireshark. The traffic that was encrypted cannot be interpreted and hence we performed the second phase of our MITM attack. In this phase, the secure channel was downgraded from Hyper Text Transfer Protocol Secure (HTTPS) to Hypertext Transfer Protocol (HTTP) without alerting the user or the application. This was done through the use of Secure Socket Layer Strip (SSLStrip). This attack takes advantage of the weak implementation of HTTPS. The attacker intercepts the TLS secure authentication sent from the server intended for the victim. The attacker replaces any HTTPS link with HTTP (including 302 redirects) and sends it to the victim. In parallel, it maintains a secure HTTPS connection with the server. In this way, as seen in figure 4 [22], the attacker communicates with the victim using HTTP and the server using HTTPS.

To protect against such protocol downgrade attacks, HTTP Strict Transport Security (HSTS) was introduced in 2010. HSTS tells the browser to only communicate with the server using secure HTTPS and never over insecure HTTP. Developers can make use of this header and also preload their domain in a list called HSTS preload, which is maintained by Google and shared by various modern browsers including Chrome, Safari, Opera, IE11, Edge and Firefox. Due to this implementation, the SSL Strip attack is rendered useless since the browser is forced to connect via HTTPS even if HTTP is specified.

HSTS was bypassed using a vulnerability in its implementation since HSTS is usually preloaded on a per-host-name basis. HTTPS links were downgraded to HTTP and were then

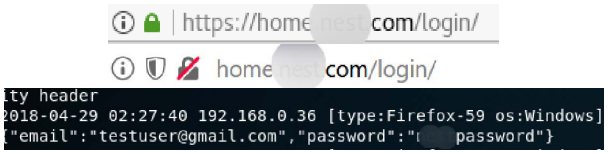

Fig. 5. Attack Vector 1 - SSL Stripped

prefixed with a custom URL. As a result, this newly generated URL did not get picked up by the HSTS preload list and went through unblocked. Since this link did not point to a real host server, it needed to be resolved back to the real server but this was done easily using a custom Domain Name System (DNS) server since a MITM attack was already set up.

Another MITM attack was performed that makes the use of an HTTPS proxy to exploit Server Name Indication (SNI) callbacks. SNI is an extension to the TLS computer networking protocol by which a client indicates which host name it is attempting to connect to at the start of the handshaking process. There is a vulnerability because the server is allowed to present multiple certificates on a single IP address allowing multiple HTTPS websites without requiring the same certificate. The attacker catches the host name in the up-stream SNI callback. The callback is paused, attack connects to the server and fetches the certificate. The attacker then signs the certificate using his own Certificate Authority (CA) and passes it to the client. In this way, the attacker was able to establish an connection with the client and any encrypted traffic can be decrypted by the attacker since it was signed using the attack's own certificate.

Exploiting these vulnerabilities, two broad attacks vectors were performed. Attack vector 1 used IP forwarding, sniffing, ARP spoofing, SSLStrip+ and DNS2Proxy. Attack vector 2 used IP forwarding, sniffing, ARP spoofing, HTTPS Proxy. The results of these attacks are described in the following section.

VI. Results

Extensive penetration testing was performed using the attack vectors mentioned in the implementation section. Various web and mobile IoT applications were attacked during this test. These applications are spread across a wide variety of domains such as smart homes, security systems, health-care and connected cars. These applications have a high impact in terms of the damage caused if compromised. Most of the applications considered in this study have a user base of over 1 million and high potential impact if vulnerable.

As discussed above, attack vector 1 used IP forwarding, sniffing, ARP spoofing, SSLStrip+ and DNS2Proxy. This enabled the attacker to downgrade the secure SSL connection to an insecure connection. Once executed, the application became insecure and this vulnerability was exploited by sniffing through the packets revealing sensitive information such as user-names and passwords.

As shown in figure 5, this major home automation organization loads its information using HTTPS by default.

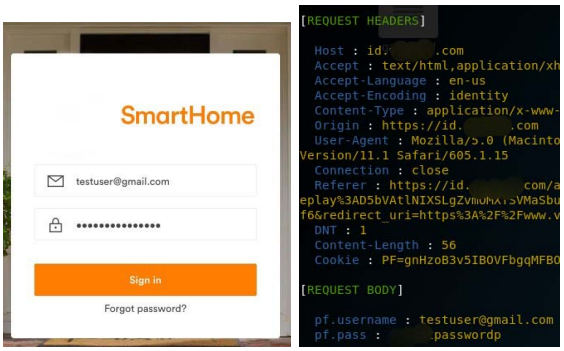

Fig. 6. Attack Vector 2 - HTTPS Proxy

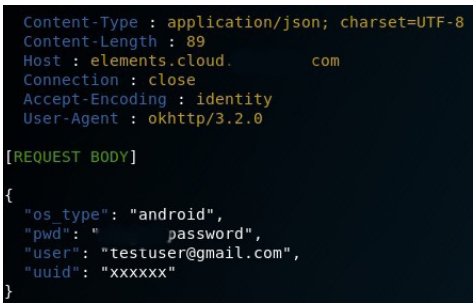

Fig. 7. Logs with credentials in plain-text

Once executed, the connection between the attacker and the application gets stripped of its HTTPS and loads with the insecure HTTP. This, in turn, results in the extraction of credentials from the application.

Attack vector 2 used IP forwarding, sniffing, ARP spoofing and HTTPS Proxy which enabled the attacker to pretend that the connection is legitimate and secure by signing its own certificate. In this case, the user did not get any hint and entered sensitive information into the application which was easily extracted in clear-text at the attacker's end.

As shown in figure 6, this popular smart home security systems organization displays a secure connection link which has been set up by the attacker. The user-name and password are displayed in clear text.

A total of 28 applications were attacked. Tables I and II provide the snapshot of all applications tested and how successful the attacks were in the study. Successful attacks include being able to extract sensitive login data (figure 7)

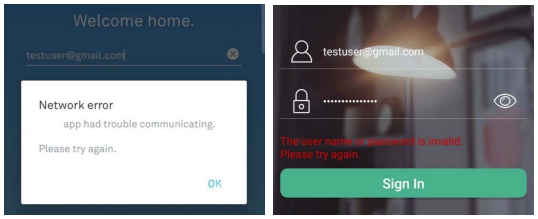

Fig. 8. Smart home applications

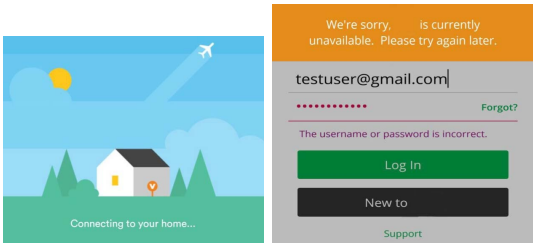

Fig. 9. Security system applications

TABLE I
ATTACK VECTOR 1 TEST RESULT

	Successful	Failed
Smart Home	6	10
Security	2	1
Health Care	1	2
Connected Cars	4	2

and DoS attacks (figure 8 and 9).

These results show how vulnerable a lot of the popular IoT applications are. Though many have attempted to secure their link over HTTPS, it is noticed that in a lot of cases security is compromised due to misconfiguration of security parameters. This is a matter of great concern and needs awareness.

VII. COUNTER-MEASURES

The vulnerabilities described above need to be resolved in a way that the security of these applications can be restored. Some counter-measures to strengthen IoT security based on the above results are provided below [23].

- Ensure proper configuration of applications as per official documentation. While installing the application server, it is advised not to use features that are not required. These might leave open ports and back-doors for attackers.
- Configure a well-formed HSTS header on your application server. There are 3 distinct parameters that are to be included in the HSTS response header for the security to be proper. These are
  - max-age: the amount of time in seconds the header should stay alive. Should be a large value greater than 10368000 seconds (120 days) and ideally to 31536000 (one year).
  - includeSubDomains: this states to require HSTS on all subdomains. RFC 6797, section 14.4 advocates that a web application must aim to add the includeSubDomain directive in the policy definition whenever possible. The directives presence ensures

TABLE II
ATTACK VECTOR 2 TEST RESULT

	Successful	Failed
Smart Home	15	1
Security	3	0
Health Care	3	0
Connected Cars	6	0

the HSTS policy is applied to the domain of the issuing host and all of its subdomains.
  - preload: this gives your authorization to be included in preload lists
- Providing an HSTS header on all HTTPS resources in the target domain.
- It is not recommended to provide an HSTS policy via the http-equiv attribute of a meta tag.
- Encrypt sensitive data such as passwords at the application side before sending the request to the server. This keeps the password secure even if the connection gets compromised.
- The application should never send an HSTS header over a plain-text HTTP header, as doing so makes the connection vulnerable to SSL stripping attacks.
- Use certificate/public key pinning for applications to prevent from HTTPS proxy attacks.

Besides these, some generic guidelines that every developer should keep in mind while securing these applications are:

- Apply intrusion detection at all layers in order to catch suspicious activities at any of the three layers [24].
- Error messages should be short and generic and should not supply any additional information to the user. Extra information in the error logs can enable better formulation of the kind of attacks that could be planted for that specific application.
- Secure applications from Cross-Site Scripting attacks and SQL Injection is to have extensive input validations in various form fields.
- Ensure risk management for maintaining the security of the system in light of new threats by applying software patches and updates regularly [25].
- Apply a trust management system for the whole IoT system to ensure the trustworthiness in terms of meeting security goals across the system [3].

Organizations should invest in security and pen-testing before they launch their application to the market. It is better to find vulnerabilities before going live otherwise it could cause millions of dollars in damage in addition to the loss of reputation and credibility. These countermeasures are bound to strengthen application security and prevent breaches in the future.

VIII. Conclusion

After analyzing the results of the study that was conducted over various IoT applications, it is evident that many IoT applications are highly vulnerable. These applications have a high impact on the lives of end-users and thus their security is critical and should be taken seriously. The study also shows a clear lack of awareness and effort towards IoT application security. It is important to note that the misconfiguration of parameters was a big factor in exposing the whole system to be vulnerable to various attacks. The counter-measures mentioned above would safeguard them from similar attacks, thus making the application more robust and secure.

References

[1] Insights, O. E. C. D., and Increase Revenue. "The Internet of things." (2017).

[2] The 10 most popular Internet of Things applications right now. Retrieved from https://iot-analytics.com/10-internet-of-things-applications

[3] I. Andrea, C. Chrysostomou, and G. Hadjichristofi, Internet of Things: Security vulnerabilities and challenges, 2015 IEEE Symposium on Computers and Communication (ISCC), 2015.

[4] N Kshetri, "An Opinion on the" Report on Securing and Growing the Digital Economy"." *IEEE Security & Privacy 15*, no. 1, pp. 80-85, 2017.

[5] Z.-K. Zhang, M. C. Y. Cho, C.-W. Wang, C.-W. Hsu, C.-K. Chen, and S. Shieh, IoT Security: Ongoing Challenges and Research Opportunities, *2014 IEEE 7th International Conference on Service-Oriented Computing and Applications*, 2014.

[6] W. Shang, Q. Ding, A. Marianantoni, J. Burke, and L. Zhang, Securing building management systems using named data networking, *IEEE Network*, vol. 28, no. 3, pp. 5056, 2014.

[7] J. Liu, Y. Xiao, and C. P. Chen, Authentication and Access Control in the Internet of Things, *2012 32nd International Conference on Distributed Computing Systems Workshops*, 2012.

[8] J. Zaddach, L. Bruno, A. Francillon, and D. Balzarotti, Avatar: A Framework to Support Dynamic Security Analysis of Embedded Systems Firmwares, *Proceedings 2014 Network and Distributed System Security Symposium*, 2014.

[9] D.Davidson, B.Moench, S.Jha, and T.Ristenpart. "FIE on Firmware: Finding Vulnerabilities in Embedded Systems Using Symbolic Execution," in *USENIX Security Symposium*, August 2013.

[10] R. Acharya, and K. Asha, "Data integrity and intrusion detection in wireless sensor networks." *2008 16th IEEE International Conference on Networks*. IEEE, 2008.

[11] B. Niu, X. Zhu, H. Chi, and H. Li, "Privacy and Authentication Protocol for Mobile RFID Systems." *Wireless Personal Communications 77*, no. 3, pp. 1713-1731, 2014.

[12] I. Andrea, C. Chrysostomou, and G. Hadjichristofi, Internet of Things: Security vulnerabilities and challenges, *2015 IEEE Symposium on Computers and Communication (ISCC)*, 2015.

[13] L. Atzori, A. Iera, and G. Morabito, "The internet of things: A survey." *Computer networks 54*, no. 15, pp. 2787-2805, 2010.

[14] R. P. Padhy, M. R. Patra, S. C. Satapathy, "Cloud Computing: Security Issues and Research Challenges", *International Journal of Computer Science and Information Technology Security (IJCSITS)*, vol. 1, no. 2, pp. 136-146, 2011.

[15] R. Roman, J. Zhou, and J. Lopez, "On the features and challenges of security and privacy in distributed internet of things." *Computer Networks 57*, no. 10, pp. 2266-2279, 2013.

[16] C.M. Medaglia, A. Serbanati, "An overview of privacy and security issues in the internet of things" in *The Internet of Things, New York:Springer*, pp. 389-395, 2010.

[17] S. Sicari, A. Rizzardi A, L.A. Grieco, A. Coen-Porisini, "Security, privacy and trust in Internet of Things: The road ahead." *Computer networks 76*, pp. 146-164, 2015.

[18] F.A. Alaba, M. Othman, I.A.T. Hashem, F. Alotaibi, "Internet of Things security: A survey." *Journal of Network and Computer Applications 88*, PP. 10-28, 2017.

[19] R. Coulter, and L. Pan, "Intelligent agents defending for an IoT world: A review." *Computers & Security 73*, pp. 439-458, 2018.

[20] D. Wichers, "Owasp top-10 2013", *OWASP Foundation*, February 2013.

[21] Anon. Retrieved May 4, 2018 from https://www.incapsula.com/web-application-security/man-in-the-middle-mitm.html

[22] Introduction to MITM with SSLStrip. (2016, March 31). Retrieved from https://hack-ed.net/2016/03/31/introduction-to-mitm-with-sslstrip/

[23] P. Sexton, HSTS — HTTP strict transport security. [online] Varvy.com. Available at: https://varvy.com/pagespeed/hsts.html [Accessed 10 Mar. 2019]

[24] A. Patcha, J. M. Park, "An overview of anomaly detection techniques: Existing solutions and latest technological trends", *Computer Networks*, vol. 51, no. 12, pp. 3448-347, 2007.

[25] C. Liu, Y. Zhang, J. Zeng, L. Peng, R Chen, "Research on Dynamical Security Risk Assessment for the Internet of Things inspired by immunology", *Natural Computation (ICNC) 2012 Eighth International Conference*, pp. 874-878, 2012.